

ghzfirodz

February 25, 2023

```
[507]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from google.colab import files

import os
```

```
[508]: df = pd.read_csv("netflix.csv")
```

```
[509]: df.columns
```

```
[509]: Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
          'release_year', 'rating', 'duration', 'listed_in', 'description'],
          dtype='object')
```

```
[510]: df.index
```

```
[510]: RangeIndex(start=0, stop=8807, step=1)
```

```
[511]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   show_id         8807 non-null   object
1   type            8807 non-null   object
2   title           8807 non-null   object
3   director        6173 non-null   object
4   cast            7982 non-null   object
5   country         7976 non-null   object
6   date_added      8797 non-null   object
7   release_year    8807 non-null   int64
8   rating          8803 non-null   object
9   duration        8804 non-null   object
10  listed_in       8807 non-null   object
```

```

11 description    8807 non-null    object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB

```

```
[512]: df['date_added'] = pd.to_datetime(df['date_added'])
```

```
[513]: df.release_year.min()
```

```
[513]: 1925
```

```
[514]: df.release_year.max()
```

```
[514]: 2021
```

```
[515]: df.shape
```

```
[515]: (8807, 12)
```

```
[516]: df.describe(include='all')
```

```

<ipython-input-516-174ba9bf1a5c>:1: FutureWarning: Treating datetime data as
categorical rather than numeric in `.describe` is deprecated and will be removed
in a future version of pandas. Specify `datetime_is_numeric=True` to silence
this warning and adopt the future behavior now.
df.describe(include='all')

```

```
[516]:
```

	show_id	type	title	director \
count	8807	8807	8807	6173
unique	8807	2	8807	4528
top	s1	Movie	Dick Johnson Is Dead	Rajiv Chilaka
freq	1	6131	1	19
first	NaN	NaN	NaN	NaN
last	NaN	NaN	NaN	NaN
mean	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN

	cast	country	date_added	release_year \
count	7982	7976	8797	8807.000000
unique	7692	748	1714	NaN
top	David Attenborough	United States	2020-01-01 00:00:00	NaN
freq	19	2818	110	NaN
first	NaN	NaN	2008-01-01 00:00:00	NaN
last	NaN	NaN	2021-09-25 00:00:00	NaN

mean	NaN	NaN	NaN	2014.180198
std	NaN	NaN	NaN	8.819312
min	NaN	NaN	NaN	1925.000000
25%	NaN	NaN	NaN	2013.000000
50%	NaN	NaN	NaN	2017.000000
75%	NaN	NaN	NaN	2019.000000
max	NaN	NaN	NaN	2021.000000

	rating	duration	listed_in \
count	8803	8804	8807
unique	17	220	514
top	TV-MA	1 Season	Dramas, International Movies
freq	3207	1793	362
first	NaN	NaN	NaN
last	NaN	NaN	NaN
mean	NaN	NaN	NaN
std	NaN	NaN	NaN
min	NaN	NaN	NaN
25%	NaN	NaN	NaN
50%	NaN	NaN	NaN
75%	NaN	NaN	NaN
max	NaN	NaN	NaN

	description
count	8807
unique	8775
top	Paranormal activity at a lush, abandoned prope...
freq	4
first	NaN
last	NaN
mean	NaN
std	NaN
min	NaN
25%	NaN
50%	NaN
75%	NaN
max	NaN

```
[517]: df['country']=df['country'].astype('category')
df['listed_in']=df['listed_in'].astype('category')
```

```
[518]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
#   ...
```

```

---  -----
0  show_id      8807 non-null  object
1  type         8807 non-null  object
2  title        8807 non-null  object
3  director     6173 non-null  object
4  cast         7982 non-null  object
5  country      7976 non-null  category
6  date_added   8797 non-null  datetime64[ns]
7  release_year 8807 non-null  int64
8  rating       8803 non-null  object
9  duration     8804 non-null  object
10 listed_in    8807 non-null  category
11 description  8807 non-null  object
dtypes: category(2), datetime64[ns](1), int64(1), object(8)
memory usage: 764.8+ KB

```

```

[519]: for i in df.columns:
        null_rate = df[i].isna().sum() / len(df) * 100
        if null_rate >= 0 :
            print("{} null rate: {}".format(i,round(null_rate,2)))

```

```

show_id null rate: 0.0%
type null rate: 0.0%
title null rate: 0.0%
director null rate: 29.91%
cast null rate: 9.37%
country null rate: 9.44%
date_added null rate: 0.11%
release_year null rate: 0.0%
rating null rate: 0.05%
duration null rate: 0.03%
listed_in null rate: 0.0%
description null rate: 0.0%

```

```

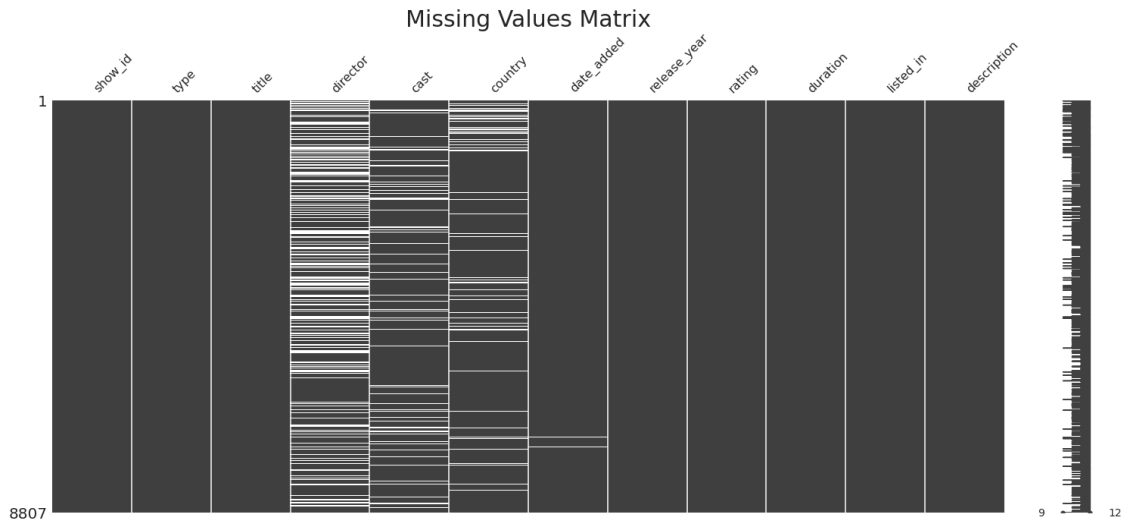
[520]: import missingno as msno
        msno.matrix(df)
        plt.title("Missing Values Matrix",fontsize=30)

```

```

[520]: Text(0.5, 1.0, 'Missing Values Matrix')

```



```
[521]: df.director.value_counts()
```

```
[521]: Rajiv Chilaka                19
       Raúl Campos, Jan Suter       18
       Marcus Raboy                 16
       Suhas Kadav                  16
       Jay Karas                    14
       ..
       Raymie Muzquiz, Stu Livingston  1
       Joe Menendez                  1
       Eric Bross                    1
       Will Eisenberg               1
       Mozez Singh                   1
       Name: director, Length: 4528, dtype: int64
```

```
[522]: df.director.nunique()
```

```
[522]: 4528
```

```
[523]: df['cast']=df['cast'].str.split(',')
       df['listed_in']=df['listed_in'].str.split(',')
       df['country']=df['country'].str.split(',')
       df['director']=df['director'].str.split(',')
       
```

```
[524]: df = df.explode("cast")
       df = df.explode("country")
       df = df.explode("director")
       df = df.explode("listed_in").reset_index(drop=True)
       df['cast'] = df['cast'].str.strip()
       df['listed_in'] = df['listed_in'].str.strip()
```

```
df['director'] = df['director'].str.strip()
df['country'] = df['country'].str.strip()
```

```
[525]: df.shape
```

```
[525]: (202065, 12)
```

BEFORE DATA CLEANING

```
[526]: for i in df.columns:
        null_rate = df[i].isna().sum() / len(df) * 100
        if null_rate >= 0 :
            print("{} null rate: {}".format(i,round(null_rate,2)))
```

```
show_id null rate: 0.0%
type null rate: 0.0%
title null rate: 0.0%
director null rate: 25.06%
cast null rate: 1.06%
country null rate: 5.89%
date_added null rate: 0.08%
release_year null rate: 0.0%
rating null rate: 0.03%
duration null rate: 0.0%
listed_in null rate: 0.0%
description null rate: 0.0%
```

```
[527]: df['date_added'] = df.groupby(['release_year','type'])['date_added'].
        ↪transform(lambda x: x.fillna(x.mode().iloc[0]))
```

```
[528]: df['country'] = df.groupby(['listed_in'])['country'].transform(lambda x: x.
        ↪fillna(x.mode().iloc[0]))
```

```
[529]: df['director'] = df.groupby(['release_year'])['director'].transform(lambda x: x.
        ↪fillna(x.mode()[0]) if len(x.mode()) > 0 else x)
```

```
[530]: df['cast'] = df.groupby(['release_year'])['cast'].transform(lambda x: x.
        ↪fillna(x.mode()[0]) if len(x.mode()) > 0 else x)
```

```
[531]: df = df.dropna(subset=['duration',])
```

```
[532]: df.drop(df[(df['release_year'] == 1925) & df.isnull().any(axis=1)].index,
        ↪inplace=True)
```

```
/usr/local/lib/python3.8/dist-packages/pandas/core/frame.py:4906:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
return super().drop()
```

```
[533]: df['day_of_week'] = df['date_added'].dt.day_name()
```

```
[534]: df['rating'].fillna("Not Rated",inplace=True)
df["year_added"] = df['date_added'].dt.year
df["year_added"] = df["year_added"].astype("Int64")
df["month_added"] = df['date_added'].dt.month
df["month_added"] = df["month_added"].astype("Int64")
```

AFTER DATA CLEANING”

```
[535]: for i in df.columns:
        null_rate = df[i].isna().sum() / len(df) * 100
        if null_rate >= 0 :
            print("{} null rate: {}".format(i,round(null_rate,2)))
```

```
show_id null rate: 0.0%
type null rate: 0.0%
title null rate: 0.0%
director null rate: 0.0%
cast null rate: 0.0%
country null rate: 0.0%
date_added null rate: 0.0%
release_year null rate: 0.0%
rating null rate: 0.0%
duration null rate: 0.0%
listed_in null rate: 0.0%
description null rate: 0.0%
day_of_week null rate: 0.0%
year_added null rate: 0.0%
month_added null rate: 0.0%
```

```
[536]: movies = df[df['type'] == 'Movie']
tv_shows = df[df['type'] == 'TV Show']

# create two subplots for movies and tv shows
fig, axes = plt.subplots(nrows=2, figsize=(5, 10))
sns.distplot(movies['release_year'], ax=axes[0])
sns.distplot(tv_shows['release_year'], ax=axes[1])

# set titles and labels for the subplots
axes[0].set_title('Frequency of Movies Released Till Now')
axes[1].set_title('Frequency of TV Shows Released Till Now')
axes[0].set_xlabel('Release Year')
axes[1].set_xlabel('Release Year')
```

```
axes[0].set_ylabel('Density')
axes[1].set_ylabel('Density')

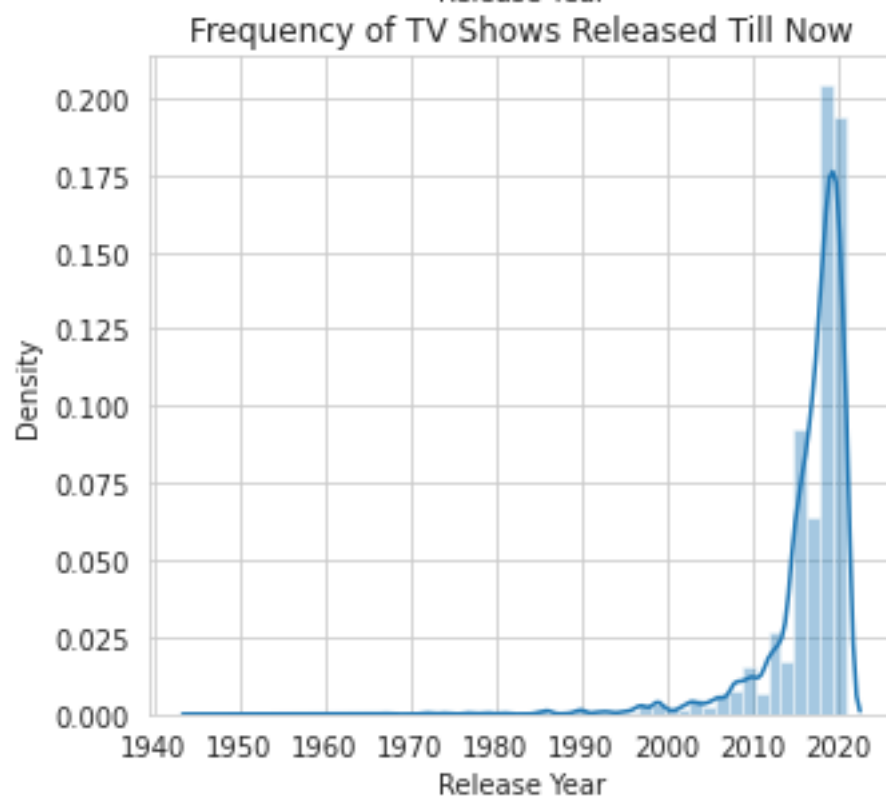
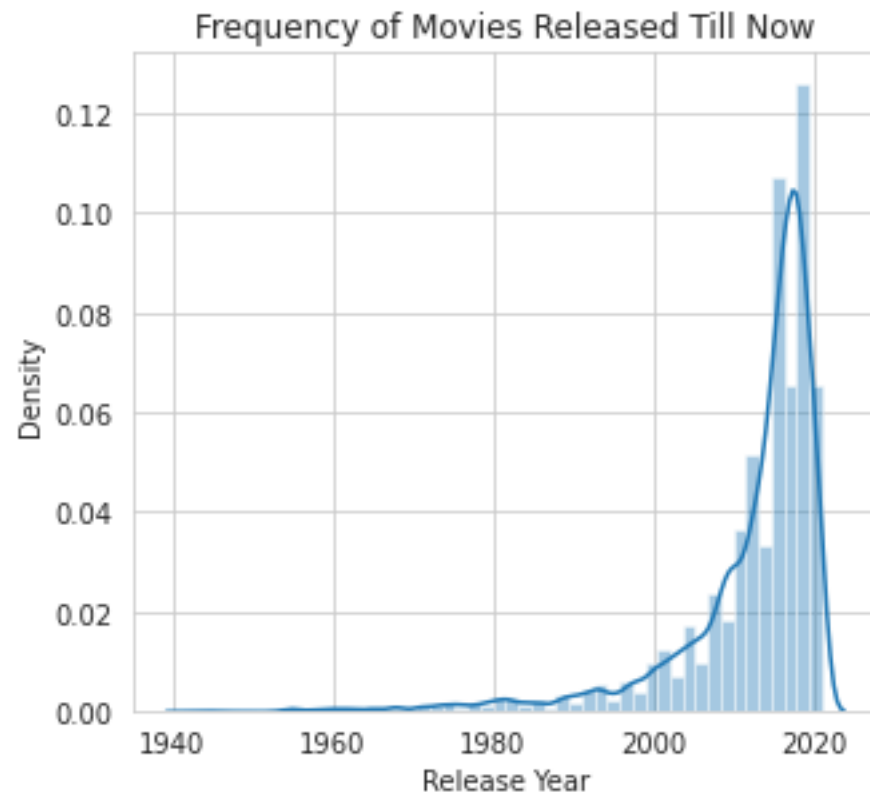
plt.show()
```

```
/usr/local/lib/python3.8/dist-packages/seaborn/distributions.py:2619:
FutureWarning: `distplot` is a deprecated function and will be removed in a
future version. Please adapt your code to use either `displot` (a figure-level
function with similar flexibility) or `histplot` (an axes-level function for
histograms).
```

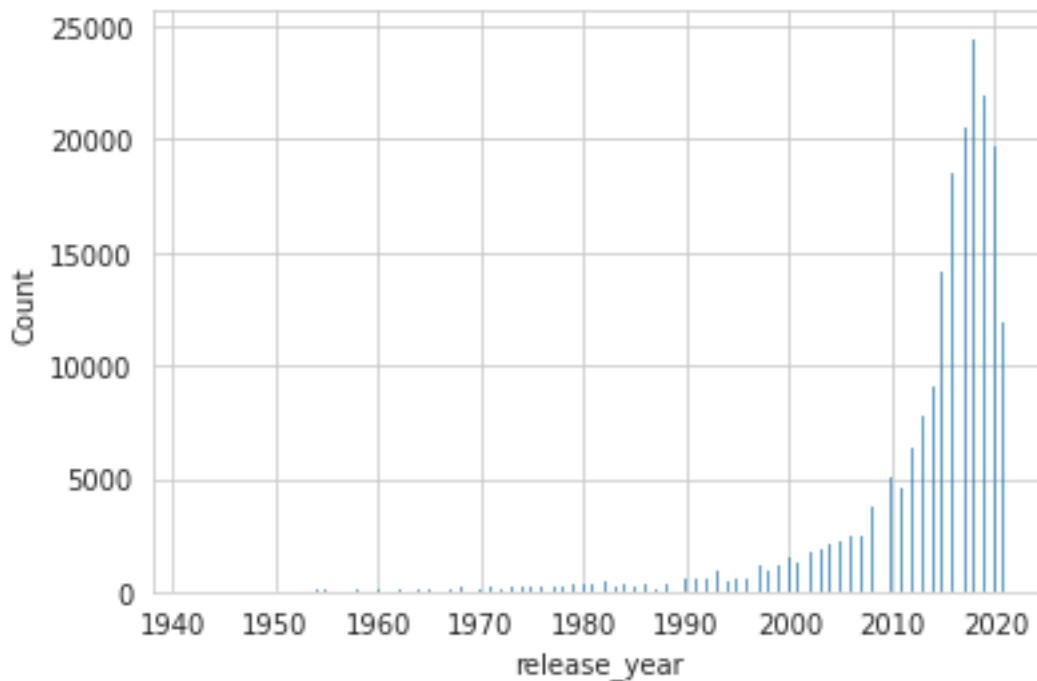
```
    warnings.warn(msg, FutureWarning)
```

```
/usr/local/lib/python3.8/dist-packages/seaborn/distributions.py:2619:
FutureWarning: `distplot` is a deprecated function and will be removed in a
future version. Please adapt your code to use either `displot` (a figure-level
function with similar flexibility) or `histplot` (an axes-level function for
histograms).
```

```
    warnings.warn(msg, FutureWarning)
```

```
[537]: ax = sns.histplot(data=df, x="release_year")
plt.show()
```



```
[538]: df1 = df.loc[df['type'] == 'Movie']
df1['duration'] = df1['duration'].str.replace(" min", "").astype("float").
    .astype("int")
df1['duration'].value_counts()
sns.boxplot(data=df1, x="duration")
df1['duration'].max()
df1['duration'].describe()
```

<ipython-input-538-4ed8347f45a5>:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

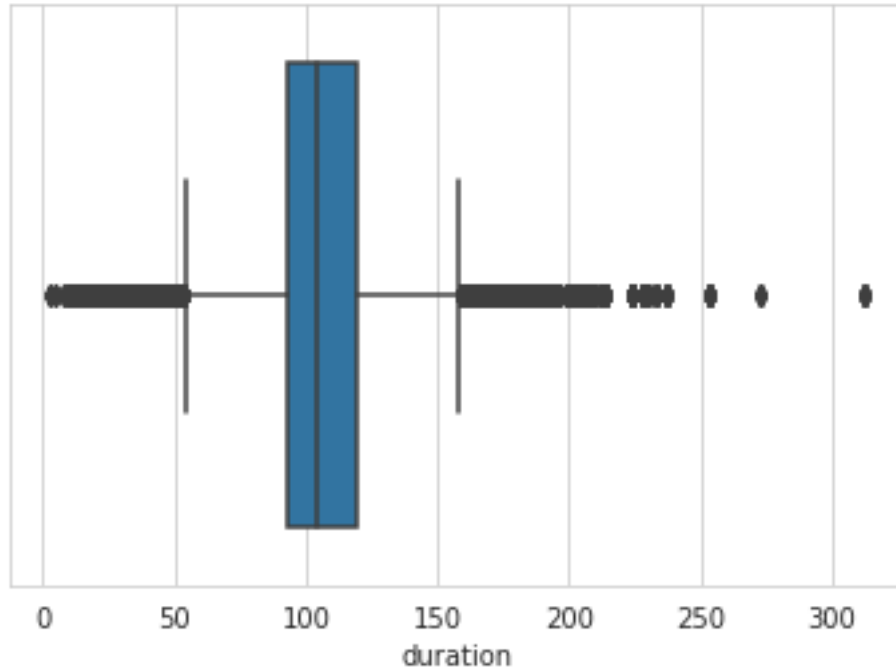
```
df1['duration'] = df1['duration'].str.replace("
min", "").astype("float").astype("int")
```

```
[538]: count    145914.000000
mean      106.840385
std       24.709395
min        3.000000
25%       93.000000
```

```

50%          104.000000
75%          119.000000
max           312.000000
Name: duration, dtype: float64

```



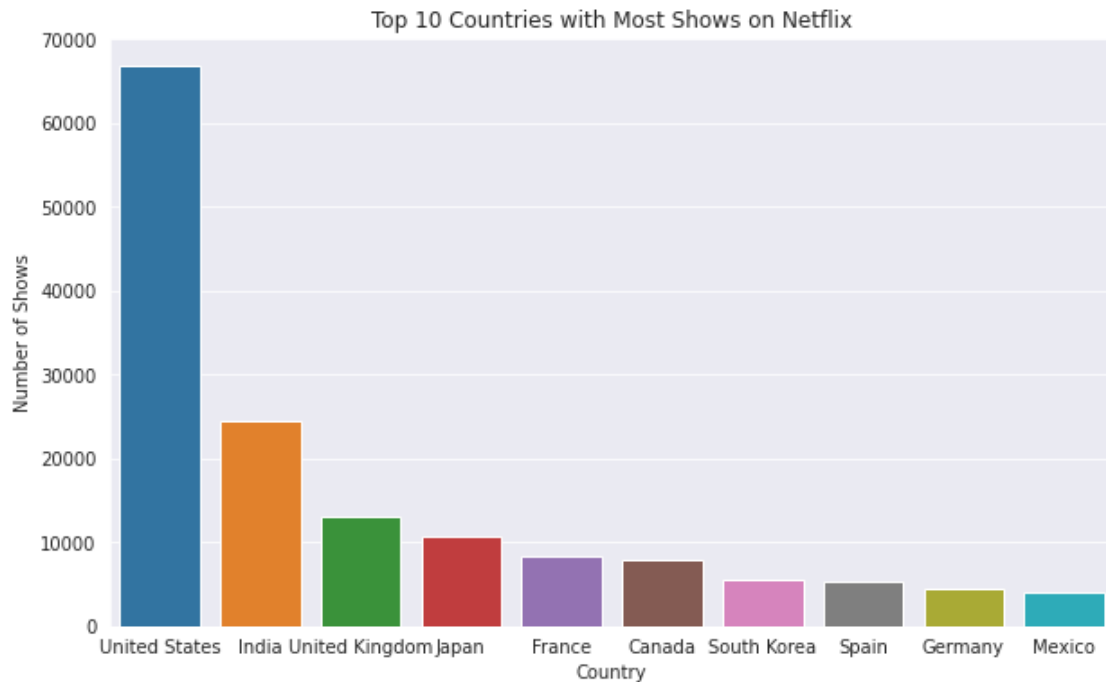
```
[539]: df1.loc[df1['duration']==df1['duration'].max()].iloc[0]['title']
```

```
[539]: 'Black Mirror: Bandersnatch'
```

```
[540]: df1.loc[df1['duration']==df1['duration'].min()].iloc[0]['title']
```

```
[540]: 'Silent'
```

```
[541]: country_grouped = df.groupby('country').count().sort_values(by='show_id',
    ↪ascending=False)[:10]
country_grouped
sns.set_style('darkgrid')
plt.figure(figsize=(10, 6))
sns.barplot(x=country_grouped.index, y='show_id', data=country_grouped)
plt.title('Top 10 Countries with Most Shows on Netflix')
plt.xlabel('Country')
plt.ylabel('Number of Shows')
plt.show()
country_grouped.index
```



```
[541]: Index(['United States', 'India', 'United Kingdom', 'Japan', 'France', 'Canada',
           'South Korea', 'Spain', 'Germany', 'Mexico'],
          dtype='object', name='country')
```

#How has the number of movies released per year changed over the last 20-30 years?

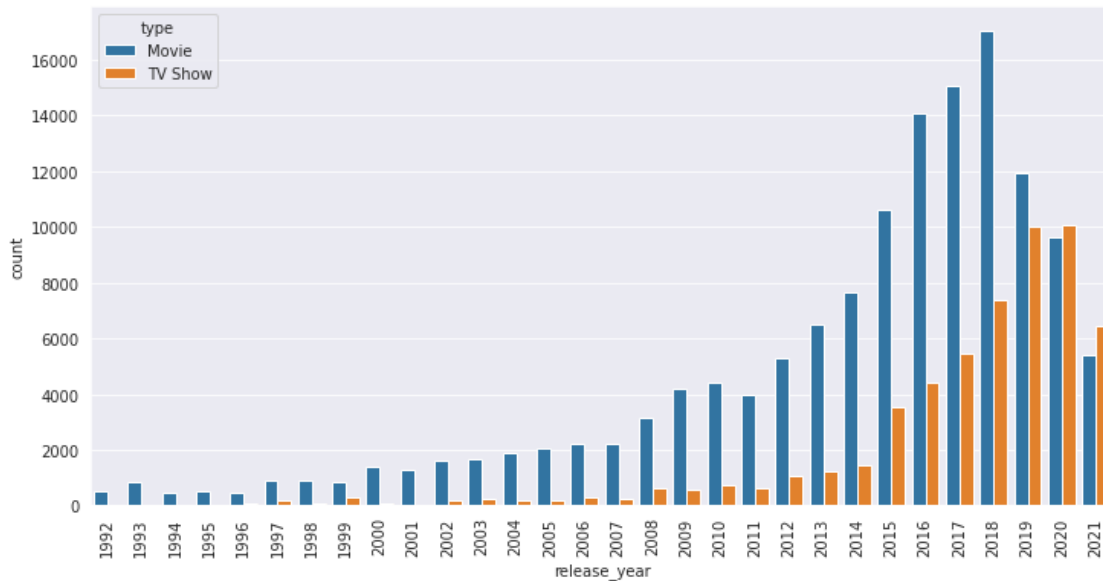
```
[542]: df1 = df.loc[df['release_year'] > df['release_year'].max() - 30,
                  ↪['release_year', 'type']]
```

```
[543]: df1['type'].value_counts()
```

```
[543]: Movie      138820
       TV Show    55797
       Name: type, dtype: int64
```

```
[544]: # create the count plot
       plt.figure(figsize=(12, 6))
       plt.xticks(rotation=90)
       sns.countplot(x='release_year', hue='type', data=df1)
```

```
[544]: <AxesSubplot:xlabel='release_year', ylabel='count'>
```



```
[545]: rating_groupby = df.groupby(['rating', 'country'])['title'].count()
india_ratings = rating_groupby.loc[(slice(None), 'India')].
    ↪sort_values(ascending=False)
india_ratings = india_ratings.iloc[:10]
india_ratings = india_ratings.rename_axis(['rating']).reset_index()
india_ratings

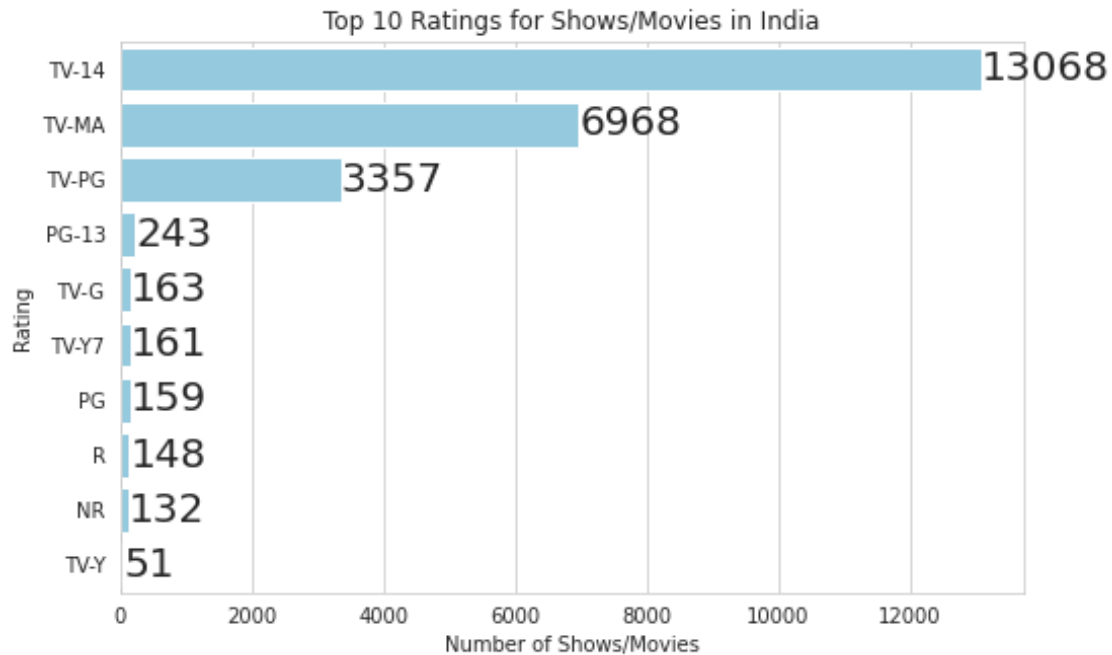
sns.set_style('whitegrid')
sns.set_color_codes('pastel')

fig, ax = plt.subplots(figsize=(8, 5))
sns.barplot(x='title', y='rating', data=india_ratings, color='skyblue', ax=ax)

ax.set_title('Top 10 Ratings for Shows/Movies in India')
ax.set_xlabel('Number of Shows/Movies')
ax.set_ylabel('Rating')

# Add count as text
for i, v in enumerate(india_ratings['title']):
    ax.annotate(str(v), xy=(v + 1, i), va='center', fontsize=20)

plt.show()
```



```
[546]: rating_groupby = df.groupby(['rating', 'country'])['title'].count()
general_audience = rating_groupby.loc['G', (slice(None))].
    ↳ sort_values(ascending=False)
general_audience = general_audience.iloc[:10]
general_audience
```

```
[546]: rating  country
G         United States    907
         United Kingdom   130
         Spain            74
         Ireland          65
         Germany          56
         Canada           48
         France           36
         East Germany     30
         Japan            30
         West Germany     30
Name: title, dtype: int64
```

```
[547]: genre_groupby = df.groupby(['listed_in', 'country'])['title'].count()
general_audience = genre_groupby.loc['International Movies'].
    ↳ sort_values(ascending=False)
general_audience = general_audience.iloc[:10]
general_audience
```

```
[547]: country
      India      8657
      France    1740
      United States 1211
      United Kingdom 1199
      Spain     1131
      Japan     810
      Egypt     787
      Nigeria    784
      Indonesia  727
      Turkey     712
      Name: title, dtype: int64
```

```
[548]: type_groupby = df.groupby(['type','country'])['title'].count()
      type_groupby = type_groupby.loc['TV Show'].sort_values(ascending=False)
      type_groupby
```

```
[548]: country
      United States    16478
      Japan           7039
      United Kingdom   4417
      South Korea      4344
      Mexico           2221
      ...
      Cyprus           10
               8
      Greece           6
      Belarus          6
      Uruguay          3
      Name: title, Length: 66, dtype: int64
```

Explanation:- For the last 30 The Gradual Rate of movies has always been increasing but there was a dip from the year 2018

#Comparison of tv shows vs. movies

```
[549]: import seaborn as sns
      import matplotlib.pyplot as plt

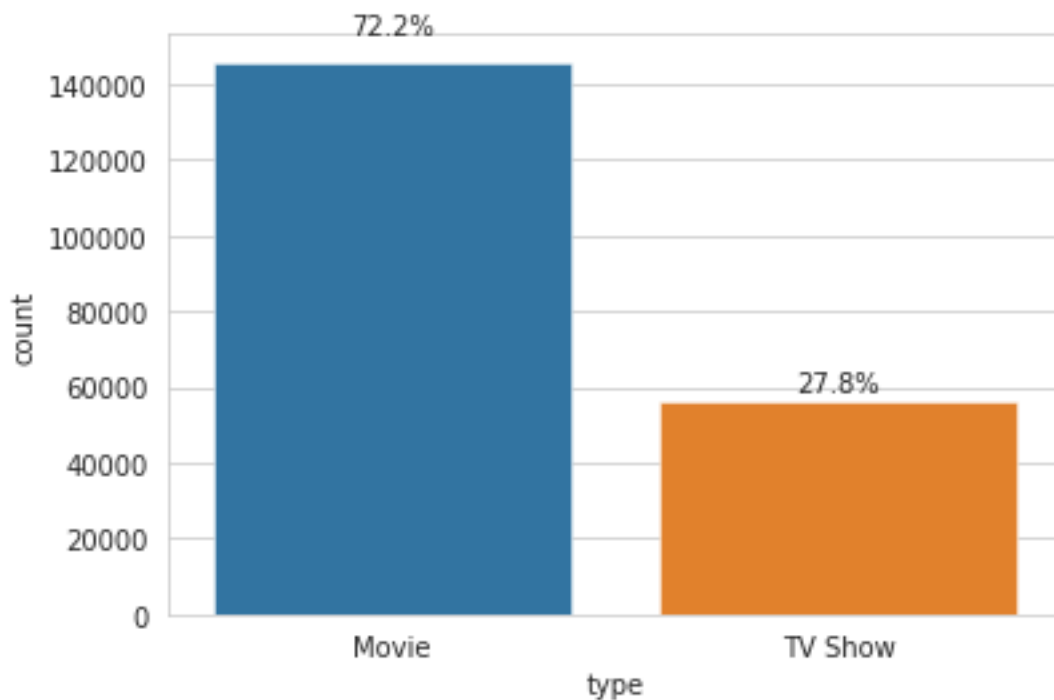
      # create countplot
      ax = sns.countplot(x='type', data=df, linewidth=0.5)

      # get counts and calculate percentages
      counts = df['type'].value_counts()
      total = counts.sum()
      percentages = counts / total * 100
      print("percentages",percentages)
```

```
# add percentage labels
for i, p in enumerate(ax.patches):
    height = p.get_height()
    width = p.get_width()
    x, y = p.get_xy()
    ax.text(x + width/2 , y + height * 1.05, f'{percentages[i]:.1f}%',
    ↪ha='center')

plt.show()
```

```
percentages Movie      72.212847
TV Show      27.787153
Name: type, dtype: float64
```



Netflix has more number of Movies than **TV**

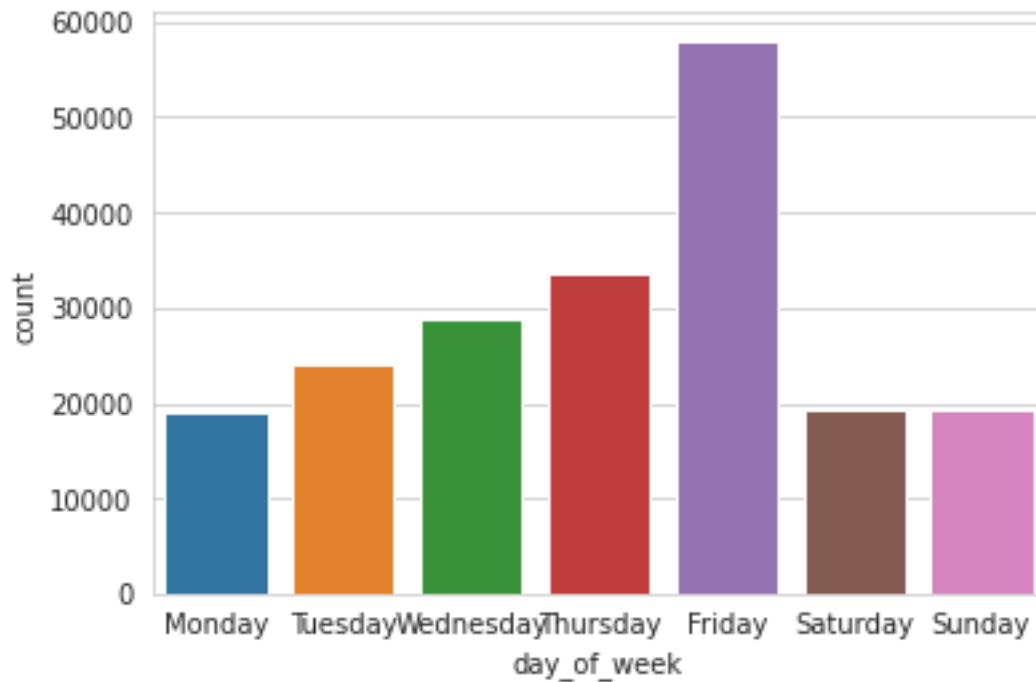
#What was the best time to launch a TV show?

```
[550]: df['day_of_week'] = df['date_added'].dt.day_name()
df1 = df.loc[df['type']=='TV Show']
```

```
[551]: sns.countplot(x=df["day_of_week"], order=["Monday", "Tuesday", "Wednesday",
    ↪ "Thursday", "Friday", "Saturday", "Sunday"])
```



```
[551]: <AxesSubplot:xlabel='day_of_week', ylabel='count'>
```



The Best Time To Launch A TV show would be Friday**

```
[552]: data_corr = df.corr()  
data_corr
```

```
[552]:
```

	release_year	year_added	month_added
release_year	1.000000	0.053252	-0.032102
year_added	0.053252	1.000000	-0.167706
month_added	-0.032102	-0.167706	1.000000

```
[553]: sns.heatmap(data = df.corr())
```

```
[553]: <AxesSubplot:>
```



1 Analysis of Actors and Directors

```
[554]: df['duration'].value_counts()
```

```
[554]: 1 Season      35034
      2 Seasons    9559
      3 Seasons    5084
      94 min       4343
      106 min      4040
      ...
      3 min         4
      5 min         3
      11 min        2
      8 min         2
      9 min         2
      Name: duration, Length: 220, dtype: int64
```

```
[555]: # Count the number of directors in each category
movie_directors = df[df['type'] == 'Movie']['director'].nunique()
tv_directors = df[df['type'] == 'TV Show']['director'].nunique()
both_directors = df.groupby('director')['type'].nunique().eq(2).sum()

# Calculate the percentages
```

```

total_directors = df['director'].nunique()
movie_directors_percent = 100 * movie_directors / total_directors
tv_directors_percent = 100 * tv_directors / total_directors
both_directors_percent = 100 * both_directors / total_directors

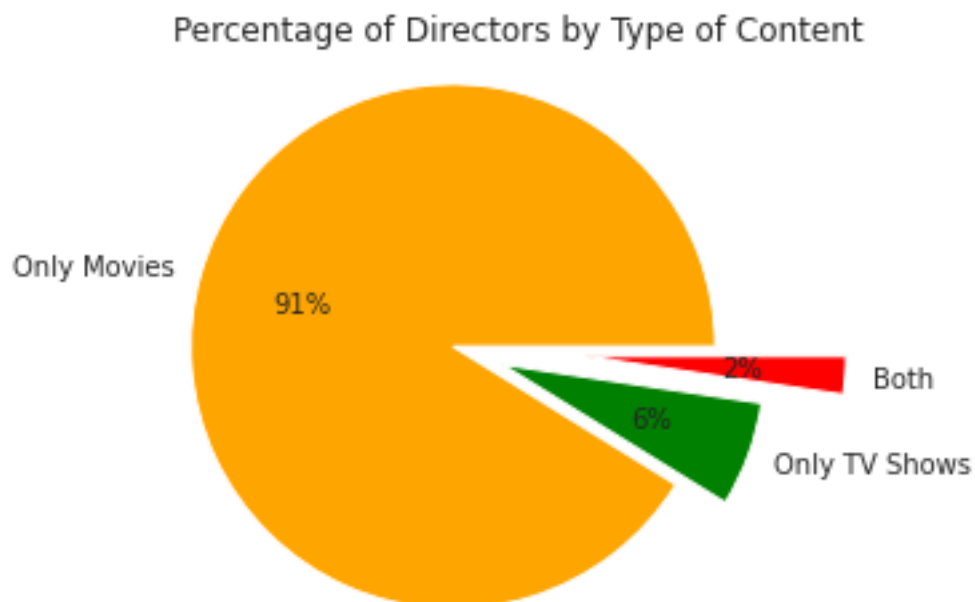
# Print the results
print(f"Percentage of directors who directed only movies:␣
↪{movie_directors_percent:.2f}%")
print(f"Percentage of directors who directed only TV shows:␣
↪{tv_directors_percent:.2f}%")
print(f"Percentage of directors who directed both movies and TV shows:␣
↪{both_directors_percent:.2f}%")
explode = [0, 0.2, 0.5]
labels = ['Only Movies', 'Only TV Shows', 'Both']
sizes = [movie_directors_percent, tv_directors_percent, both_directors_percent]
colors = ['orange', 'green', 'red']
plt.pie(sizes, labels=labels, colors=colors, explode=explode, autopct='%.0f%%')
plt.axis('equal')
plt.title('Percentage of Directors by Type of Content')
plt.show()

```

Percentage of directors who directed only movies: 95.67%

Percentage of directors who directed only TV shows: 6.79%

Percentage of directors who directed both movies and TV shows: 2.46%



```
[556]: # Count the number of cast members in each category
movie_cast = df[df['type'] == 'Movie']['cast'].nunique()
tv_cast = df[df['type'] == 'TV Show']['cast'].nunique()
both_cast = df.groupby('cast')['type'].nunique().eq(2).sum()

# Calculate the percentages
total_cast = df['cast'].nunique()
movie_cast_percent = 100 * movie_cast / total_cast
tv_cast_percent = 100 * tv_cast / total_cast
both_cast_percent = 100 * both_cast / total_cast

# Print the results
print(f"Percentage of cast members who acted only in movies:␣
↳{movie_cast_percent:.2f}%")
print(f"Percentage of cast members who acted only in TV shows: {tv_cast_percent:
↳.2f}%")
print(f"Percentage of cast members who acted in both movies and TV shows:␣
↳{both_cast_percent:.2f}%")

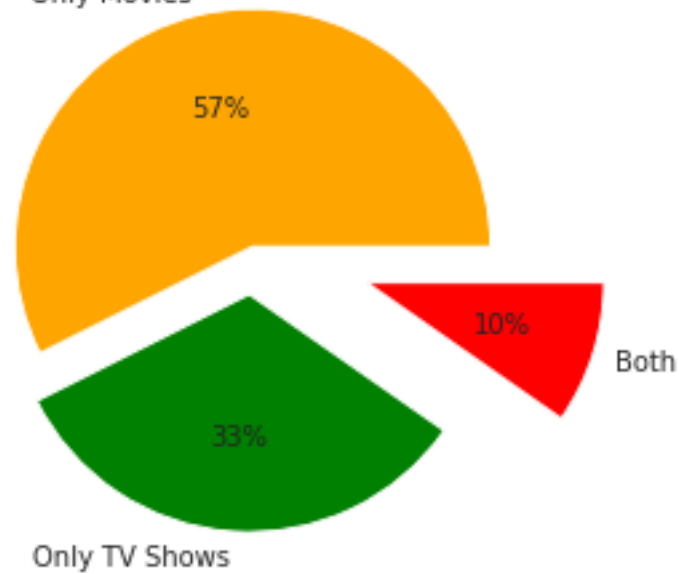
explode = [0, 0.2, 0.5]
labels = ['Only Movies', 'Only TV Shows', 'Both']
sizes = [movie_cast_percent, tv_cast_percent, both_cast_percent]
colors = ['orange', 'green', 'red']
plt.pie(sizes, labels=labels, colors=colors, explode=explode, autopct='%.0f%%')
plt.axis('equal')
plt.title('Percentage of Cast Members by Type of Content')
plt.show()
```

Percentage of cast members who acted only in movies: 71.22%

Percentage of cast members who acted only in TV shows: 40.82%

Percentage of cast members who acted in both movies and TV shows: 12.04%

Percentage of Cast Members by Type of Content



2 Does Netflix has more focus on TV Shows than movies in recent years

```
[557]: df1 = df.loc[df['release_year'] > df['release_year'].max() - 10,
↳ ['release_year', 'type']]
df1
```

```
[557]:
```

	release_year	type
0	2020	Movie
1	2021	TV Show
2	2021	TV Show
3	2021	TV Show
4	2021	TV Show
...	...	...
202060	2015	Movie
202061	2015	Movie
202062	2015	Movie
202063	2015	Movie
202064	2015	Movie

[154267 rows x 2 columns]

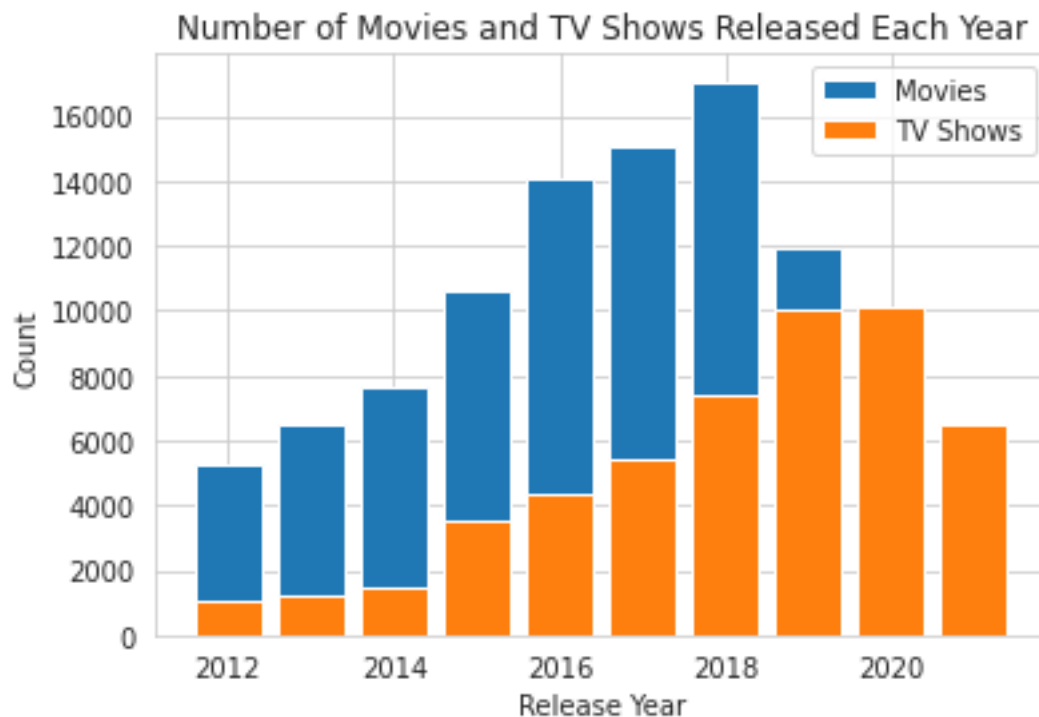
```
[558]: import matplotlib.pyplot as plt
```

```
grouped = df1.groupby(['release_year', 'type']).size().reset_index(name='count')

# Create separate DataFrames for movies and TV shows
movies = grouped[grouped['type'] == 'Movie']
tv_shows = grouped[grouped['type'] == 'TV Show']

# Plot the count of movies and TV shows over time
plt.bar(movies['release_year'], movies['count'], label='Movies')
plt.bar(tv_shows['release_year'], tv_shows['count'], label='TV Shows')

plt.xlabel('Release Year')
plt.ylabel('Count')
plt.title('Number of Movies and TV Shows Released Each Year')
plt.legend()
plt.show()
```



**YES NETFLIX HAS BEEN FOCUSING MORE ON TV SHOWS THAN MOVIES THE THE GROWTH RATE HAS ALWAYS BEEN IN A POSITIVE WAY FOR TV SHOWS THE DOWN-FALL OF MOVIES STARTED IN 2018 which shows more audience are connected to TV SHOWS THAN MOVIES

Understanding what content is available in different countries

```
[559]: df.head()
```

```
[559]: show_id      type      title      director      cast \
0      s1      Movie  Dick Johnson Is Dead      Kirsten Johnson  Adil Dehbi
1      s2      TV Show      Blood & Water  Rathindran R Prasad  Ama Qamata
2      s2      TV Show      Blood & Water  Rathindran R Prasad  Ama Qamata
3      s2      TV Show      Blood & Water  Rathindran R Prasad  Ama Qamata
4      s2      TV Show      Blood & Water  Rathindran R Prasad  Khosi Ngema

      country date_added  release_year rating  duration \
0  United States 2021-09-25      2020  PG-13      90 min
1  South Africa 2021-09-24      2021  TV-MA  2 Seasons
2  South Africa 2021-09-24      2021  TV-MA  2 Seasons
3  South Africa 2021-09-24      2021  TV-MA  2 Seasons
4  South Africa 2021-09-24      2021  TV-MA  2 Seasons

      listed_in      description \
0      Documentaries  As her father nears the end of his life, filmm...
1  International TV Shows  After crossing paths at a party, a Cape Town t...
2      TV Dramas  After crossing paths at a party, a Cape Town t...
3      TV Mysteries  After crossing paths at a party, a Cape Town t...
4  International TV Shows  After crossing paths at a party, a Cape Town t...

      day_of_week  year_added  month_added
0      Saturday      2021      9
1      Friday      2021      9
2      Friday      2021      9
3      Friday      2021      9
4      Friday      2021      9
```

```
[560]: import pandas as pd

# assuming your DataFrame is stored in a variable called 'df'
pivoted_df = pd.pivot_table(df, index='country', columns='listed_in',
                             values='title', aggfunc='count')
pivoted_df = pivoted_df.fillna(0)
pivoted_df['Total_Movies'] = pivoted_df.sum(axis=1)

pivoted_df = pivoted_df.sort_values(by='Total_Movies', ascending=False)
top_10_countries = pivoted_df.head(10)
bottom_10_countries = pivoted_df.tail(10)

top_10_countries = top_10_countries.drop('Total_Movies', axis=1)
bottom_10_countries = bottom_10_countries.drop('Total_Movies', axis=1)

bottom_10_countries
```

```

[560]: listed_in      Action & Adventure  Anime Features  Anime Series  \
country
Armenia              0.0              0.0              0.0
Palestine            0.0              0.0              0.0
Panama               0.0              0.0              0.0
Afghanistan          0.0              0.0              0.0
Mongolia             0.0              0.0              0.0
Botswana             0.0              0.0              0.0
Samoa                0.0              0.0              0.0
Uganda               0.0              0.0              0.0
Kazakhstan           0.0              0.0              0.0
Nicaragua            0.0              0.0              0.0

listed_in      British TV Shows  Children & Family Movies  Classic & Cult TV  \
country
Armenia              0.0              0.0              0.0
Palestine            0.0              0.0              0.0
Panama               0.0              0.0              0.0
Afghanistan          0.0              0.0              0.0
Mongolia             0.0              0.0              0.0
Botswana             0.0              0.0              0.0
Samoa                0.0              0.0              0.0
Uganda               0.0              0.0              0.0
Kazakhstan           0.0              1.0              0.0
Nicaragua            0.0              0.0              0.0

listed_in      Classic Movies  Comedies  Crime TV Shows  Cult Movies  ...  \
country
Armenia              0.0        0.0              0.0        0.0  ...
Palestine            0.0        0.0              0.0        0.0  ...
Panama               0.0        0.0              0.0        0.0  ...
Afghanistan          0.0        0.0              0.0        0.0  ...
Mongolia             0.0        0.0              0.0        0.0  ...
Botswana             0.0        0.0              0.0        0.0  ...
Samoa                0.0        0.0              0.0        0.0  ...
Uganda               0.0        0.0              0.0        0.0  ...
Kazakhstan           0.0        0.0              0.0        0.0  ...
Nicaragua            0.0        0.0              0.0        0.0  ...

listed_in      TV Action & Adventure  TV Comedies  TV Dramas  TV Horror  \
country
Armenia              0.0              0.0              0.0              0.0
Palestine            0.0              0.0              0.0              0.0
Panama               0.0              0.0              0.0              0.0
Afghanistan          0.0              0.0              0.0              0.0
Mongolia             0.0              0.0              0.0              0.0
Botswana             0.0              0.0              0.0              0.0

```


Samoa	0.0	0.0	0.0	0.0
Uganda	0.0	0.0	0.0	0.0
Kazakhstan	0.0	0.0	0.0	0.0
Nicaragua	0.0	0.0	0.0	0.0

listed_in country	TV Mysteries	TV Sci-Fi & Fantasy	TV Shows	TV Thrillers \
Armenia	0.0	0.0	0.0	0.0
Palestine	0.0	0.0	0.0	0.0
Panama	0.0	0.0	0.0	0.0
Afghanistan	0.0	0.0	0.0	0.0
Mongolia	0.0	0.0	0.0	0.0
Botswana	0.0	0.0	0.0	0.0
Samoa	0.0	0.0	0.0	0.0
Uganda	0.0	0.0	0.0	0.0
Kazakhstan	0.0	0.0	0.0	0.0
Nicaragua	0.0	0.0	0.0	0.0

listed_in country	Teen TV Shows	Thrillers
Armenia	0.0	0.0
Palestine	0.0	0.0
Panama	0.0	0.0
Afghanistan	0.0	0.0
Mongolia	0.0	0.0
Botswana	0.0	0.0
Samoa	0.0	0.0
Uganda	0.0	0.0
Kazakhstan	0.0	0.0
Nicaragua	0.0	0.0

[10 rows x 42 columns]

```
[561]: import matplotlib.pyplot as plt

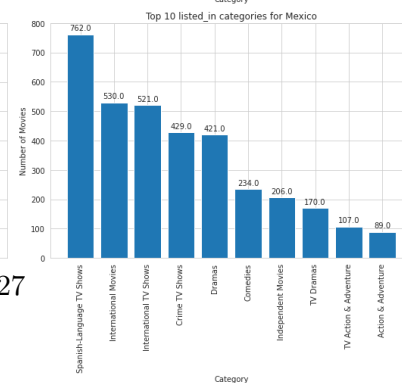
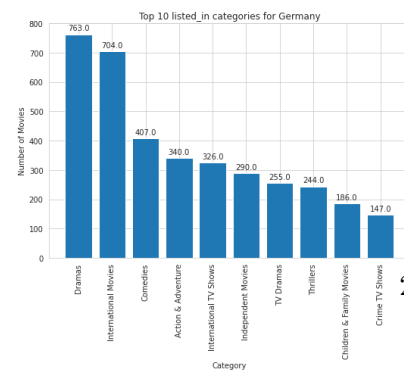
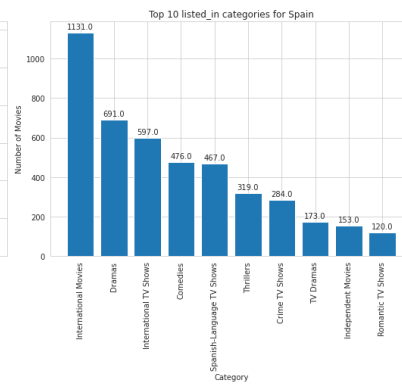
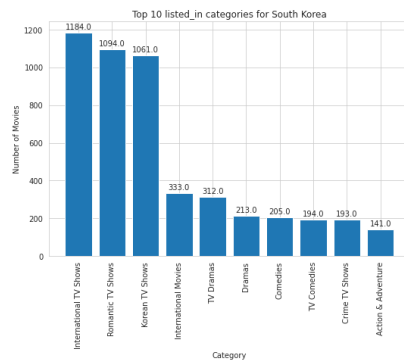
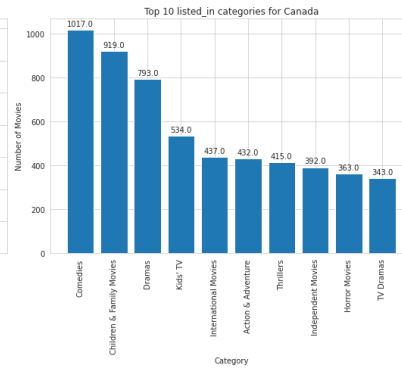
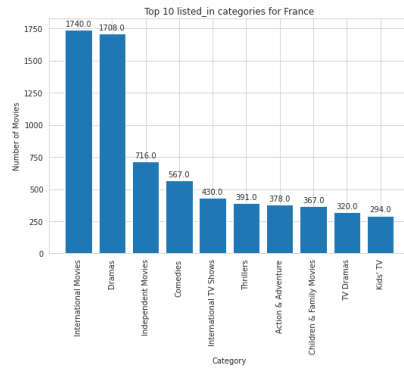
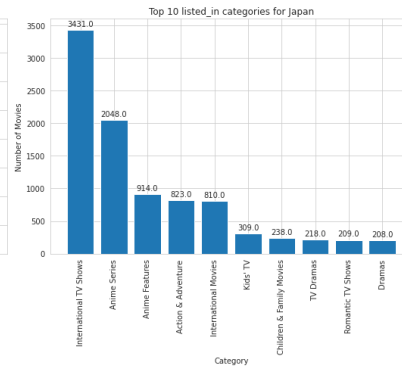
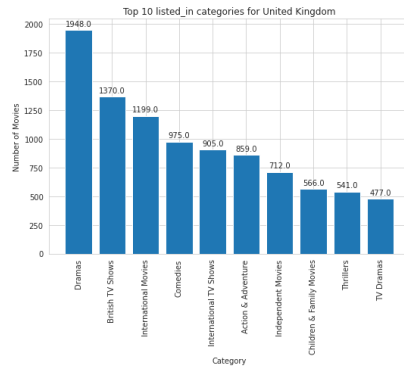
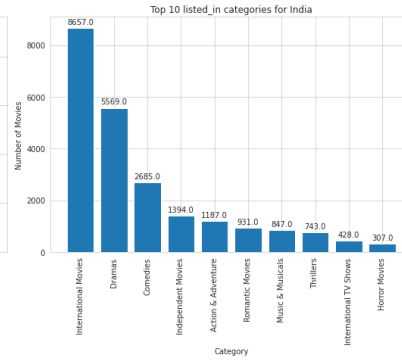
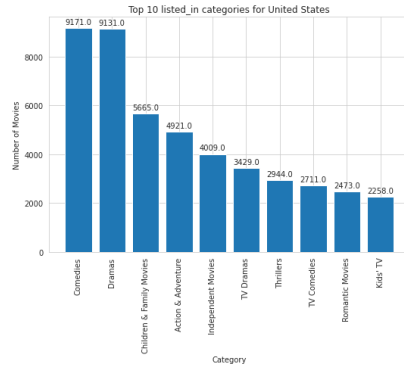
# Drop the 'Total_Movies' column from the DataFrame

fig, axes = plt.subplots(nrows=5, ncols=2, figsize=(15, 35))
for i, country in enumerate(top_10_countries.index):
    row = i // 2
    col = i % 2
    # select the top 10 listed_in categories for the current country
    top_10_categories = top_10_countries.loc[country].
    ↪sort_values(ascending=False)[:10]
    # create a bar chart for the current country
    bars = axes[row, col].bar(top_10_categories.index, top_10_categories.values)
    axes[row, col].set_title(f'Top 10 listed_in categories for {country}')
```

```

axes[row, col].set_xlabel('Category')
axes[row, col].set_ylabel('Number of Movies')
axes[row, col].tick_params(axis='x', rotation=90)
# add count labels to each bar
for bar in bars:
    height = bar.get_height()
    axes[row, col].annotate(f'{{height}}', xy=(bar.get_x() + bar.get_width() /
↪ 2, height), xytext=(0, 3),
                                textcoords="offset points", ha='center', ↵
↪ va='bottom')
plt.tight_layout()
plt.show()

```



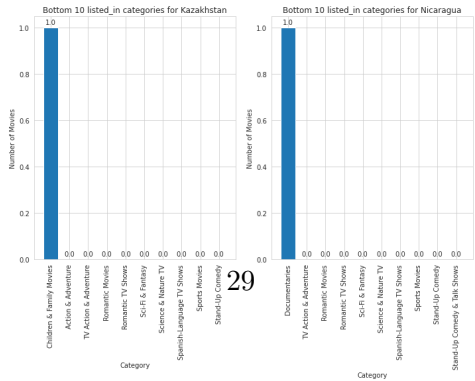
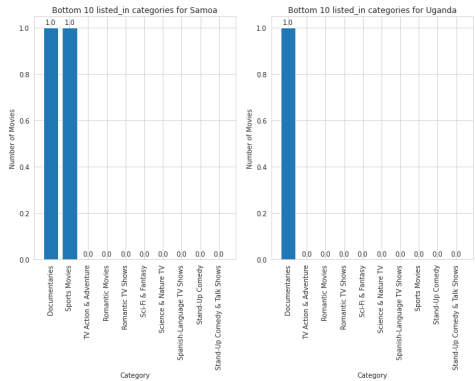
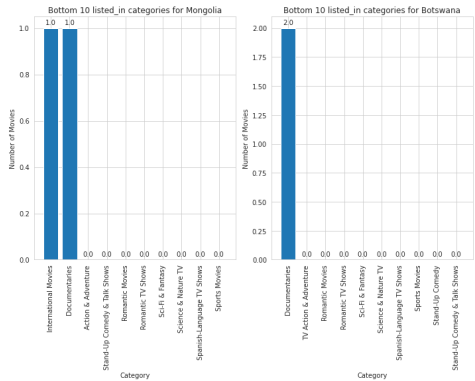
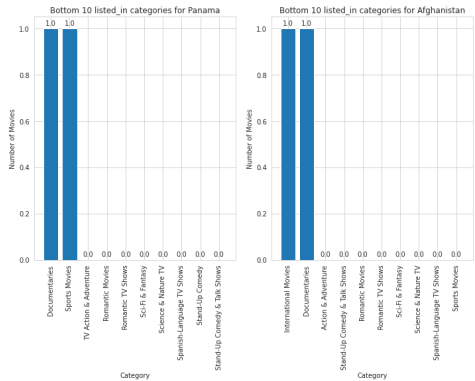
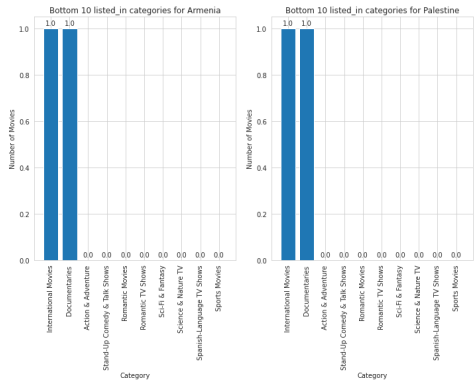
```

[562]: import matplotlib.pyplot as plt

# Drop the 'Total_Movies' column from the DataFrame

fig, axes = plt.subplots(nrows=5, ncols=2, figsize=(10, 40))
for i, country in enumerate(bottom_10_countries.index):
    row = i // 2
    col = i % 2
    # select the top 10 listed_in categories for the current country
    top_10_categories = bottom_10_countries.loc[country].
    ↪sort_values(ascending=False)[:10]
    # create a bar chart for the current country
    bars = axes[row, col].bar(top_10_categories.index, top_10_categories.values)
    axes[row, col].set_title(f'Bottom 10 listed_in categories for {country}')
    axes[row, col].set_xlabel('Category')
    axes[row, col].set_ylabel('Number of Movies')
    axes[row, col].tick_params(axis='x', rotation=90)
    # add count labels to each bar
    for bar in bars:
        height = bar.get_height()
        axes[row, col].annotate(f'{height}', xy=(bar.get_x() + bar.get_width() /
        ↪ 2, height), xytext=(0, 3),
                                textcoords="offset points", ha='center', ↪
        ↪va='bottom')
plt.tight_layout()
plt.show()

```



```
[563]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 202061 entries, 0 to 202064
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   show_id         202061 non-null object
 1   type            202061 non-null object
 2   title           202061 non-null object
 3   director        202061 non-null object
 4   cast            202056 non-null object
 5   country         202061 non-null object
 6   date_added      202061 non-null datetime64[ns]
 7   release_year    202061 non-null int64
 8   rating          202061 non-null object
 9   duration        202061 non-null object
10   listed_in       202061 non-null object
11   description     202061 non-null object
12   day_of_week     202061 non-null object
13   year_added      202061 non-null Int64
14   month_added     202061 non-null Int64
dtypes: Int64(2), datetime64[ns](1), int64(1), object(11)
memory usage: 33.1+ MB
```

```
[564]: df['cast']
```

```
[564]: 0          Adil Dehbi
      1          Ama Qamata
      2          Ama Qamata
      3          Ama Qamata
      4      Khosi Ngema
      ...
      202060      Anita Shabdish
      202061      Anita Shabdish
      202062      Chittaranjan Tripathy
      202063      Chittaranjan Tripathy
      202064      Chittaranjan Tripathy
      Name: cast, Length: 202061, dtype: object
```

```
[565]: famous_actor = df.groupby(['cast', 'country'])['title'].count().
      ↪reset_index(name='count').sort_values(by='count', ascending=False)
      famous_actor[famous_actor.country == 'India'].sort_values(by='count',
      ↪ascending=False)
```

```
[565]:
```

	cast	country	count
4517	Anupam Kher	India	113
43079	Radhika Apte	India	101
48242	Shah Rukh Khan	India	99
38286	Naseeruddin Shah	India	96
1318	Akshay Kumar	India	87
...	...	...	...
40957	Pascal Atuma	India	1
2114	Alhaji Fadika	India	1
2028	Alexei Maklakov	India	1
11839	Daniel Holguín	India	1
5680	Aída Morales	India	1

[5078 rows x 3 columns]

```
[566]: indian_actors = df[df['country'] == 'India']
famous_actor = df.groupby(['cast', 'country'])['title'].count()
max_films_actor = famous_actor.idxmax()
# # Printing the results
print(max_films_actor)
```

('Belén Cuesta', 'United States')

6.1 Comments on the range of attributes

0	show_id	8807 non-null	object
1	type	8807 non-null	object
2	title	8807 non-null	object
3	director	6173 non-null	object
4	cast	7982 non-null	object
5	country	7976 non-null	category
6	date_added	8797 non-null	datetime64[ns]
7	release_year	8807 non-null	int64
8	rating	8803 non-null	object
9	duration	8804 non-null	object
10	listed_in	8807 non-null	category
11	Description	8807 non-null	object

Country and listed_in are added as category type because they are repeated and the multiple time the main reason of converting it from object into category type is Memory efficiency, Performance improvements, Improved functionality

The date_added is converted to time stamp in order to extract month day year which helps in analysis

show_id is a unique variable

Type deals with only 2 a)MOVIE b)TV SHOW

In this dataset we deal mostly with categorical data

6.2 Comments on the distribution of the variables and relationship between them

Box plots, Countplots, Bar plots are used in this project for univariate and bivariate analysis

sns.distplot(df['release_year']) generates a histogram with the frequency of movies released in each year on the y-axis and the release year on the x-axis. The KDE plot is a smoothed curve that represents the probability distribution of the release year. We can see the concentration of movies released in the more recent years, with a peak around 2018.

6.3 Comments for each univariate and bivariate plot

Univariate analysis:

From the box plot which represents the duration in minutes we can say the average duration of movies is around 106 minutes and max duration of a movie is 312 minutes and min duration is 3 minutes

Highest Length Movie :- Black Mirror: Bandersnatch

Lowest Length Movie :- Silent

The best time to release a TV show is Friday

The frequency of Movies and TV shows are represented through dist plot The Movies growth rate is always good. But the Growth rate has been improved in the 21st century for TV shows when compared to movies.

These are the Top 10 content generating countries

'United States', 'India', 'United Kingdom', 'Japan', 'France', 'Canada', 'South Korea', 'Spain', 'Germany', 'Mexico'

Bivariate analysis:

I used Countplot to Check the growth rate of Movies and TV shows per release year Its graph tells us the Movies Growth rate had a dip in 2018 and TV shows growth rate has been steadily increasing

Pie charts were used to understand the directors who were a part of only movies,TV shows and both

Pie charts were used to understand the cast who were a part of only movies,TV shows and both

Business Insights

- INDIA's CONTENT IS MOSTLY FOCUSSED ON TV-PG: TV Parental Guidance Suggested ,TV-MA: TV Mature Audience,TV-14: TV 14 and older
- FOR General Audiences USA made more movies followed by UK and Spain
- International Movies has more demand in INDIA followed by France and UK
- Netflix has 72% content filled with movies rest deals with TV Shows
- Uruguay Country produces very less TV SHOWS Kazakhstan,Nicaragua,Uganda stands in the bottom place with the same record of producing movies
- USA,INDIA,UK are in Top3 in producing Movies and United-States,Japan,UnitedKingdom,South,Korea,Mexico are in top 5 in producing TV - SHOWS
- The Highest Movies Acted Cast is From United States and his name is Belén Cuesta has done nearly 167
- The Lowest Movies Acted Cast is From United States and his name is Belén Cuesta has done nearly 167
- Anupam Kher,Radhika Apte,Shah Rukh Khan are most famous actors in INDIA they acted in more than 90 films

Recommendations:

- More number of TV Shows should be added in the geographies like South Korea, Japan, UK.
- Most viewership of TV Shows are till 4 -5 seasons only, in almost every genre. So adding more number of seasons in each category won't provide positive result
- Developing markets like India, Japan, and Canada are content consuming both in Movies and in TV Shows market with descent enough duration. Loading more/ trying new content will make sense.r
- There are few countries where there is no response in terms

- **International Movies have high demand in almost every countries Netflix should promote more international films where its response is very less like Panama & Botswana**
- **As the Viewership is very less in those countries it should try to deliver good and famous content to its viewers slowly**
- **Netflix should try to make people engage with its platform for example it should try to make more documentaries where people know the real truth.**
- **Countries like USA INDIA South Korea are more engaging if the content is good it should try to deliver more sequels or seasons to make the audience interactive.**